

CubeTriage

A decision-driven migration engine
for legacy OLAP cubes

— *in active development* —

One exported file in — a proposed star schema, a quantified
complexity report and a human-review workbook out.
Every trap found, flagged and priced.

*Validated on Adventure Works 2019 Enterprise Edition —
120 review decisions found in ~1.5 seconds*

Malin Jacobsson

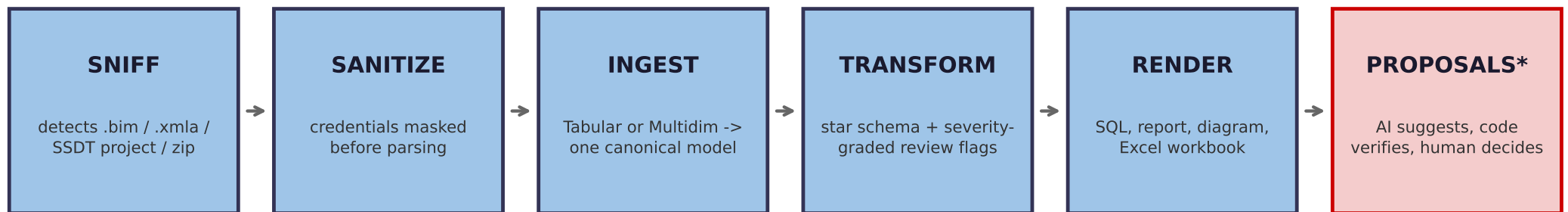
The problem

Migrating a legacy SSAS cube to a modern relational model costs a senior consultant weeks: untangling MDX, mapping role-playing dimensions, finding the KPIs and security roles nobody documented.

Before anyone can even price the migration, somebody spends days just answering: what is actually IN this cube?

CubeTriage answers that question in seconds — from a single exported metadata file. No cube data is read.

How it works



The first five stages are deterministic — same input, same output.
Proposals are opt-in behind a backend interface (mock / local LLM).

Outputs: *_canonical.json · *_out.json · *_schema.sql · *_load.sql ·
*_validation.sql · *_review.md · *_hitl.xlsx · *_diagram.png ·
*_decisions.json · *_run_report.json

Result on a real cube

Adventure Works 2019 Enterprise Edition (Multidimensional) —
Microsoft's own reference cube, built to demonstrate every
enterprise SSAS feature.

16 dimensions · 11 facts · 79 relationships · 38 measures

mapped automatically

22 calculated measures in MDX

flagged for manual translation — not ignored

29 role-playing · 4 parent-child · 4 reference · 6 m2m bridges

structural traps found

31 unmigratable constructs (KPIs, actions, perspectives)

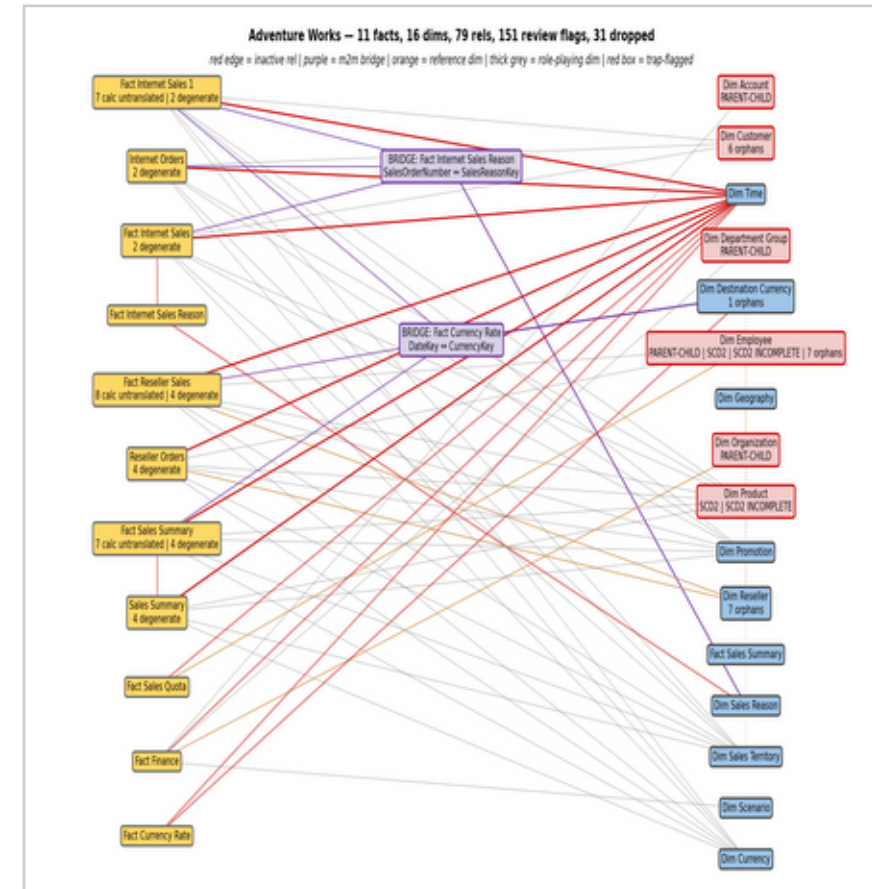
reported, never lost

101 inferred assumptions

each documented in its own channel

120 human-review items

the workbook IS the work order



Full pipeline run: ~1.5 seconds. Also verified on the
Tabular AdventureWorks .bim (compat 1400) — tables classified
purely from the relationship graph, no naming conventions needed.
Newest: generated load SQL + a 101-check validation harness.

The review loop

Reviewing IS migrating — each decision mutates the model and regenerates the artifacts, not just the report.

Every warn flag and dropped item becomes a numbered decision

decisions.json — stable ids,
survives re-runs

Click a flag — see where the problem lives

canonical source slice +
star-schema target table

Approve · reject · override — then regenerate

decided entries persist; unapplied
overrides are reported, never silent

Zero-dependency, offline, runs where the data lives

the Excel workbook stays as
the exportable record

*First run ~1.5 s — then each decision cycle
(apply + regenerate all artifacts) costs ~1 second.*

The screenshot displays the 'adventureworks' interface with a list of 115 decisions. The top bar shows '2 of 115 decided' and a 'Download decisions.json' button. The list of decisions includes:

open	approved	rejected	overridden	all
113	1	0	1	115

The list of decisions includes:

- 087AE3F orphan_attribute (Dim Customer.Date First Purchase)
- 084AE2A orphan_attribute (Dim Customer.Phone)
- 0347411 orphan_attribute (Dim Customer.Email Address)
- 0385C77 orphan_attribute (Dim Customer.CommuteDistanceSort)
- 043WF03 orphan_attribute (Dim Customer.Address Line1)
- 08NK266 parent_child_flattened (Dim Department Group)
- 043F368 orphan_attribute (Dim Destination Currency.LCID)
- 02M7791 scd2_incomplete (Dim Employee)
- 04C386F scd2_dimension (Dim Employee)
- 0027668 parent_child_flattened (Dim Employee)
- 02FA8E4 orphan_attribute (Dim Employee.Birth Date)

The detailed view of decision 087AE3F - orphan_attribute shows the source and target tables, the decision status, and options to approve, reject, or override the decision.

Source: MODEL/DIMENSIONS/DIM_CUSTOMER/ATTRIBUTES/DATE_FIRST_PURCHASE

```
{
  "name": "Date First Purchase",
  "source_column": "SimpleDateFirstPurchase",
  "source_table": "dbo_DimCustomer",
  "hidden": true,
  "orphan": true
}
```

Target: dbo_DimCustomer.Date First Purchase

Decision: CURRENT: OPEN

Buttons: Approve, Reject, Override

Comment: [Text area]

Buttons: Save comment, Reopen

Design principles

Nothing is hidden

Constructs the engine sees but cannot migrate are recorded in a dropped channel. A reviewer always sees the full picture.

Provenance is structural

Observed facts, inferred values, dropped constructs and AI proposals are four separate channels — what the engine KNOWS vs GUESSES.

AI proposes — never decides

An optional backend may suggest resolutions. Code verifiers check every proposal against the model; the human approves.

Runs where the data lives

Deterministic, offline-capable, no telemetry — designed to run as a container inside the customer's own perimeter.